

Assignment : 20.2

Name: Alli Harika

HTno: 2303A510I7

Batch: 21

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:

Generate a simple Python user registration and login system that stores usernames and passwords.

Code:

```
b 19 > 31-03-2026.py > hash_password
22 # =====
23 # TASK 1: PASSWORD SECURITY
24 # =====
25 import hashlib
26 users = {}
27 def hash_password(password):
28     return hashlib.sha256(password.encode()).hexdigest()
29 def register():
30     username = input("Enter username: ")
31     password = input("Enter password: ")
32     users[username] = hash_password(password)
33     print("User registered securely!")
34
35 def login():
36     username = input("Enter username: ")
37     password = input("Enter password:")
38
39     if users.get(username) == hash_password(password):
40         print("Login successful")
41     else:
42         print("Invalid credentials")
43
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS +

```
enter your choice: 1
enter username: harika
enter password: 1234
user registered securely!
```

```
==== AI SECURITY LAB MENU =====
```

```
7. Exit
Enter your choice: 2
Enter username: harika
Enter password:1234
Login successful
```

Explanation:

- The AI-generated code stored passwords in **plain text**.
- This is dangerous because:
 - Hackers can easily read passwords if data is leaked.
 - No protection for user credentials.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:

Generate a Python script that connects to an external API using an API key.

9 > 31-03-2026.py > ...

```
# =====  
# TASK 2: SECURE API CALL  
# =====  
import requests  
def fetch_api_data():  
    print("\nFetching API data securely...")  
    url = "https://jsonplaceholder.typicode.com/posts"  
  
    try:  
        response = requests.get(url)  
        data = response.json()  
  
        print("Data fetched successfully!\n")  
  
        # Display first 2 records  
        for post in data[:2]:  
            print("ID:", post["id"])  
            print("Title:", post["title"])  
            print("Body:", post["body"])  
            print("-" * 40)  
  
    except Exception as e:  
        print("Error:", e)
```

```
Enter your choice: 3
```

```
Fetching API data securely...
```

```
Data fetched successfully!
```

```
ID: 1
```

```
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
```

```
Body: quia et suscipit
```

```
suscipit recusandae consequuntur expedita et cum  
reprehenderit molestiae ut ut quas totam  
nostrum rerum est autem sunt rem eveniet architecto
```

```
-----
```

```
ID: 2
```

```
Title: qui est esse
```

```
Body: est rerum tempore vitae
```

```
sequi sint nihil reprehenderit dolor beatae ea dolores neque  
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
```

```
qui aperiam non debitis possimus qui neque nisi nulla
```

```
-----
```

Explanation:

- Sensitive data is not exposed in source code.
- Improves security and prevents misuse.
- Code becomes safe for sharing and deployment.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

- Refractor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:

Generate a simple Python encryption and decryption program.

```

68 # =====
69 # TASK 3: STRONG ENCRYPTION
70 # =====
71
72 from cryptography.fernet import Fernet
73
74 def encryption_demo():
75     print("\nEncryption Demo...")
76
77     key = Fernet.generate_key()
78     cipher = Fernet(key)
79     message = input("Enter message to encrypt: ")
80     encrypted = cipher.encrypt(message.encode())
81     print("Encrypted:", encrypted)
82     decrypted = cipher.decrypt(encrypted).decode()
83     print("Decrypted:", decrypted)
84 # =====

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter your choice: 4

Encryption Demo...

Enter message to encrypt: hello

Encrypted: b'gAAAAABpy_P1USxpBoqwFqhyTn32TwNtWCs_-XPVIb1fkzYKJZ27LqF56lXEfdt4AGmnBPyasnvuJM0ptUDBjQCmzYhP-jYM0A=='

Decrypted: hello

Explanation:

- Data is securely encrypted using a key.

- Without the key, decryption is not possible.
- Ensures confidentiality of sensitive data.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:

Generate a simple Flask API with login functionality.

Code:

```
84 # =====
85 # TASK 4: SIMPLE SECURE LOGIN CHECK
86 # =====
87
88 def secure_login_demo():
89     print("\nSecure Login Demo...")
90
91     stored_user = "admin"
92     stored_pass = hash_password("admin")
93
94     username = input("Enter username: ")
95     password = input("Enter password: ")
96
97     if username == stored_user and hash_password(password) == stored_pass:
98         print("Login successful (secure)")
99     else:
100         print("Invalid credentials")
101
102 # =====
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Secure Login Demo...
Enter username: admin
Enter password: admin
Login successful (secure)

Explanation:

- Improved authentication security
- Prevented exposure of credentials
- Enhanced reliability and safety of API

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:

Generate a Python program that takes user input and performs arithmetic operations.

Code:

```
lab 19 > 31-03-2026.py > encryption_demo
102  #####
103  # TASK 5: EXCEPTION HANDLING
104  # =====
105
106  def safe_calculator():
107      print("\nSafe Calculator...")
108
109      try:
110          a = int(input("Enter number 1: "))
111          b = int(input("Enter number 2: "))
112
113          print("Division:", a / b)
114
115      except ValueError:
116          print("Invalid input! Please enter numbers only.")
117
118      except ZeroDivisionError:
119          print("Cannot divide by zero!")
120
121      except Exception as e:
122          print("Unexpected error:", e)
123
124  # =====
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter your choice: 6

Safe Calculator...
Enter number 1: 10
Enter number 2: 2
Division: 5.0

powershell
Python

Explanation:

- Program no longer crashes
- Displays user-friendly error messages
- Improves reliability and user experience